OpenAI API | Anthropic API | ClarifySTL | 150-sample benchmark

# NL-STL Benchmark Bash Runbook

Copy-paste commands for setup, pilot tests, full overnight runs, and evaluation.

**Your mock test already passed.**
The dataset can be read, results can be written, and the evaluator can generate its CSV and HTML reports.

## Files assumed

```
stl_llm_eval/
  run_benchmark.py
  evaluate_results.py
  nl_stl_benchmark_150.csv
  requirements.txt
  .env.example
  benchmark_api.sh
  benchmark_clarify.sh
  run_clarify_dataset.py
  normalize_clarify_outputs.py
```

## Before running anything

1. Edit `HARNESS_DIR` at the top of both Bash scripts.
2. Edit `CLARIFY_REPO` and `CLARIFY_ORCHESTRATOR` in `benchmark_clarify.sh`.
3. Make the scripts executable.

```
cd /absolute/path/to/stl_llm_eval
chmod +x benchmark_api.sh benchmark_clarify.sh
```

**Important Bash rule:** never place a comment after a continuation backslash.
Incorrect: `--providers openai anthropic \ # comment`

# 1. API environment: OpenAI and Anthropic

## 1.1 Create the environment

```
cd /absolute/path/to/stl_llm_eval
./benchmark_api.sh setup
source .venv/bin/activate
```

## 1.2 Put keys and model names in `.env`

```
OPENAI_API_KEY=your-real-openai-key
OPENAI_MODEL=gpt-5.6-terra

# Add later when available:
ANTHROPIC_API_KEY=your-real-anthropic-key
ANTHROPIC_MODEL=claude-sonnet-5
```

> Prefer `.env` over typing API keys into the terminal. Shell exports are recorded in the current shell environment and may be easier to expose accidentally. Ensure `.env` is in `.gitignore`.

## 1.3 Check that the OpenAI key loads

```
python -c "from dotenv import load_dotenv; import os; load_dotenv(); print('OpenAI key loaded:',
bool(os.getenv('OPENAI_API_KEY')))"
```

Expected:

```
OpenAI key loaded: True
```

## 1.4 Run the free mock test

```
./benchmark_api.sh mock
```

This runs five rows with the mock provider, evaluates them, and opens:

```
evaluation/mock/report.html
```

## 2. OpenAI: one sample, pilot, and full run

### 2.1 One real OpenAI sample

```
./benchmark_api.sh one-openai
```

This command:

- prints the prompt without making an API request;
- runs sample `NLSTL150-001`;
- evaluates the result;
- opens `evaluation/first_openai/report.html`.

### 2.2 Balanced 20-sample OpenAI pilot

```
./benchmark_api.sh pilot-openai
```

The pilot includes five expected-strength cases, five limitation probes, five clarification cases, and five robotics cases.

### 2.3 Full 150-sample OpenAI run

```
./benchmark_api.sh full-openai
```

Outputs:

```
runs/openai_full.jsonl
evaluation/openai_full/detailed_scores.csv
evaluation/openai_full/summary_scores.csv
evaluation/openai_full/report.html
```

> On macOS, the script uses `caffeinate -i` for the full run so the computer does not sleep. Completed results are appended immediately. Re-running the same command resumes and skips successful rows already present in the JSONL file.

### 2.4 Watch progress in another Terminal window

```
wc -l runs/openai_full.jsonl

tail -f runs/openai_full.jsonl
```

# 3. OpenAI and Anthropic together

## 3.1 Add the Anthropic key

```
ANTHROPIC_API_KEY=your-real-anthropic-key
ANTHROPIC_MODEL=claude-sonnet-5
```

## 3.2 One sample with both providers

```
./benchmark_api.sh one-both
```

## 3.3 Full dataset with both providers

```
./benchmark_api.sh full-both
```

Outputs:

```
runs/openai_anthropic_full.jsonl
evaluation/openai_anthropic_full/detailed_scores.csv
evaluation/openai_anthropic_full/summary_scores.csv
evaluation/openai_anthropic_full/report.html
```

## Direct command equivalent

```
python run_benchmark.py \
  --dataset nl_stl_benchmark_150.csv \
  --providers openai anthropic \
  --openai-model gpt-5.6-terra \
  --anthropic-model claude-sonnet-5 \
  --concurrency 2 \
  --output runs/openai_anthropic_full.jsonl

python evaluate_results.py \
  --dataset nl_stl_benchmark_150.csv \
  --results runs/openai_anthropic_full.jsonl \
  --out-dir evaluation/openai_anthropic_full

open evaluation/openai_anthropic_full/report.html
```

# 4. ClarifySTL: configuration

ClarifySTL expects one natural-language requirement in each text file and writes a result JSON containing clarification histories and `final_stl`. The supplied runner performs the CSV-to-text and JSON-to-JSONL conversion automatically.

## 4.1 Edit paths in `benchmark_clarify.sh`

```
HARNESS_DIR="${HARNESS_DIR:-$HOME/absolute/path/to/stl_llm_eval}"
CLARIFY_REPO="${CLARIFY_REPO:-$HOME/absolute/path/to/ClarifySTL}"

CLARIFY_WORKDIR="${CLARIFY_WORKDIR:-$CLARIFY_REPO/LLM_agent_framework}"
CLARIFY_ORCHESTRATOR="${CLARIFY_ORCHESTRATOR:-$CLARIFY_WORKDIR/clarify_stl_orchestrator.py}"
```

## 4.2 Confirm the actual orchestrator filename

```
find "$CLARIFY_REPO" -name "clarify_stl_orchestrator.py"
find "$CLARIFY_REPO" -name "clarify.py"
```

Use the file that accepts:

```
--input requirement.txt
--output result.json
```

## 4.3 Create a separate ClarifySTL environment

```
./benchmark_clarify.sh setup
```

> ClarifySTL has a much larger Transformers/LLaMA-Factory dependency stack and restrictive package versions. Keep its virtual environment separate from `stl_llm_eval/.venv`.

## 4.4 Additional repository configuration

Before automated execution, configure the repository's:

- LLaMA base model;
- incompleteness detector checkpoint;
- ambiguity detector checkpoint;
- automatic responder LLM;
- final NL-to-STL transformation LLM.

The automated benchmark deliberately omits `--human`; otherwise an overnight run would pause for keyboard input.

# 5. ClarifySTL: run and evaluate

## 5.1 One sample

```
./benchmark_clarify.sh one
./benchmark_clarify.sh evaluate-one
```

Generated structure:

```
runs/clarify_one/
  requirements/NLSTL150-001.txt
  raw_json/NLSTL150-001.json
  logs/NLSTL150-001.log
  run_manifest.jsonl

runs/clarify_one.jsonl
evaluation/clarify_one/report.html
```

## 5.2 Twenty-sample pilot

```
./benchmark_clarify.sh pilot
./benchmark_clarify.sh evaluate-pilot
```

## 5.3 Full 150-sample ClarifySTL run

```
./benchmark_clarify.sh full
./benchmark_clarify.sh evaluate-full
```

Outputs:

```
runs/clarify_full/raw_json/*.json
runs/clarify_full/logs/*.log
runs/clarify_full.jsonl
evaluation/clarify_full/detailed_scores.csv
evaluation/clarify_full/summary_scores.csv
evaluation/clarify_full/report.html
```

**Comparison caveat:** OpenAI and Anthropic receive the benchmark's fixed oracle answer after asking for clarification. Standalone ClarifySTL uses its own automatic responder. Record these as different interaction conditions unless ClarifySTL is modified to consume the same oracle-answer column.

## 6. What the result files mean

| File | Purpose |
|---|---|
| `runs/*.jsonl` | Raw normalized model decisions, formulas, questions, timing, and token usage. |
| `detailed_scores.csv` | One scored row per model execution. |
| `summary_scores.csv` | Aggregated results by provider, paper target, task mode, and dataset group. |
| `report.html` | Readable summary plus the manual-review queue. |

### Evaluation buckets

| Bucket | Interpretation |
|---|---|
| automatic_pass | Correct action and canonical exact formula match, or appropriate abstention. |
| automatic_fail | Provider error, wrong action, missing formula, or invalid parsed formula. |
| manual_review | Valid but non-identical formula, uncertain semantic equivalence, or questionable clarification targeting. |

### Open a report manually on macOS

```
open evaluation/openai_full/report.html
open evaluation/clarify_full/report.html
```

### Common checks

```
# Count results
wc -l runs/openai_full.jsonl

# Inspect the latest result
tail -n 1 runs/openai_full.jsonl | python -m json.tool

# Inspect a ClarifySTL failure
cat runs/clarify_full/logs/NLSTL150-001.log
```

# 7. Minimal command checklist

## OpenAI now

```
cd /absolute/path/to/stl_llm_eval
source .venv/bin/activate
./benchmark_api.sh one-openai
./benchmark_api.sh pilot-openai
./benchmark_api.sh full-openai
```

## OpenAI plus Anthropic later

```
./benchmark_api.sh one-both
./benchmark_api.sh full-both
```

## ClarifySTL

```
./benchmark_clarify.sh setup
./benchmark_clarify.sh one
./benchmark_clarify.sh evaluate-one
./benchmark_clarify.sh pilot
./benchmark_clarify.sh evaluate-pilot
./benchmark_clarify.sh full
./benchmark_clarify.sh evaluate-full
```

**Recommended order:**
one OpenAI sample -> OpenAI pilot -> one ClarifySTL sample -> ClarifySTL pilot -> inspect both reports -> only then run all 150 rows.

---

ClarifySTL support note: its supplied transformation prompt explicitly describes future-time U, G, and F plus Boolean operators. Rows using past operators, event-edge operators, or parameterized multi-robot notation should be analyzed as unsupported-fragment challenge cases rather than ordinary in-distribution cases.